AN AI-ENABLED INTELLIGENT STUDY ASSISTANT

INDIVIDUAL THESIS

VIJITHAPALA T.G.K.G

IT22187178

LEARNING OUTCOME-AWARE EDUCATIONAL SUMMARISATION SYSTEM

R26-SE-022

B.Sc. (Hons) Degree in Information Technology
Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Vijithapala T.G.K.G

Student ID: IT22187178

Signature:  Date: **10/07/2026**

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mrs.		10/07/2026
Co-supervisor: Mr.		

ABSTRACT

University students routinely struggle to convert large volumes of lecture material into focused, exam-ready study resources. Existing AI summarization tools produce generic text abstracts that ignore the module's curriculum structure and the cognitive depth specified by each learning outcome. This thesis presents the Learning Outcome-Aware Educational Summarization System - the individual component of the Aura-Learn intelligent study assistant which addresses this gap by grounding every summary in the module's formal learning outcomes and generating pedagogically structured outputs aligned to Bloom's Taxonomy.

The pipeline combines five research contributions: (1) an LO-Anchored Hybrid Retrieval strategy that runs parallel dense-vector search (text-embedding-3-small, 1536 dimensions, Qdrant) and keyword search per learning outcome and then fuses results using reciprocal rank fusion; (2) a Cross-Encoder Re-ranking layer that re-scores retrieved chunks jointly against the query and each learning outcome using a sentence-transformers cross-encoder; (3) a Bloom's-Aware Prompt Builder that injects Bloom's taxonomy level-specific generation instructions for each learning outcome into the LLM prompt; (4) a multi-output generation engine supporting summaries, flashcards, quizzes, and mind maps with Gemini 2.0 Flash / GPT-4o as the LLM backend; and (5) a suite of four novel educational evaluation metrics — Learning Outcome Coverage Score (LOCS), Bloom's Alignment Score (BAS), Source Grounding Rate (SGR), and the composite Educational Relevance Score (ERS = 0.4·LOCS + 0.3·BAS + 0.2·SGR + 0.1·Coherence) — that measure educational quality dimensions absent from standard NLP metrics such as ROUGE and BERT Score.

Keywords — RAG, Learning Outcome Alignment, Bloom's Taxonomy, Educational Summarisation, Hybrid Retrieval, Cross-Encoder Re-ranking, ERS, LOCS, BAS, SGR, FastAPI, Qdrant, PostgreSQL

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mrs. Suriya Kumari, for her continuous guidance, invaluable feedback, and unwavering encouragement throughout the entire development of this thesis. Her expertise in intelligent tutoring systems and AI-enhanced learning has been instrumental in shaping the research direction of this component.

I also extend my genuine appreciation to my co-supervisor, Mr. Ravi Supunya, for his constructive suggestions, technical insights, and steady support throughout the implementation and evaluation phases.

My heartfelt thanks go to my project group members of R26-SE-022, whose collaboration on shared infrastructure, API contract design, and system integration made the full Aura Learn platform possible.

Finally, I am deeply grateful to the Sri Lanka Institute of Information Technology (SLIIT) for providing the academic environment, resources, and cloud infrastructure access that enabled this research.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 1
1.3 Problem statement 2
1.4 Research gap 2
1.5 Aim and objectives 2
1.6 Research questions or hypotheses 3
1.7 Scope, limitations and assumptions 3
1.8 Individual contribution 4
1.9 Thesis structure 4
2 LITERATURE REVIEW AND RELATED WORK 6
2.1 Review method and scope 6
2.2 Domain and theoretical foundations 6
2.3 Critical review of directly relevant prior work 6
2.4 Existing systems, products and techniques 7
2.5 Gap synthesis and conceptual framework 7
3 RESEARCH METHODOLOGY 9
3.1 Research design 9
3.2 Data, participants or study materials 9
3.3 Experimental or study procedure 10
3.4 Measures and evaluation metrics 11
3.5 Data analysis 11
3.6 Ethics, privacy, security and responsible AI 11
3.7 Reproducibility and research artefacts 12
4 PRODUCT AND SOLUTION DESIGN 13
4.1 Requirements 13
4.2 Component architecture within the complete solution 13
4.3 Design alternatives and decisions 14
4.4 Detailed design 14
4.5 Standards and professional practice 15
5 IMPLEMENTATION, INTEGRATION AND TESTING 17
5.1 Development environment and technology stack 17

5.2 Component implementation 17
5.3 Integration with the other components 18
5.4 Verification and software testing 19
5.5 Deployment and operation 19
6 RESULTS AND VALIDATION 21
6.1 Research results 21
6.2 Product and technical results 22
6.3 User or stakeholder validation 22
6.4 Integration results 22
6.5 Summary of findings 23
7 DISCUSSION 24
7.1 Interpretation of findings 24
7.2 Comparison with literature baselines and existing solutions 24
7.3 Research contribution 25
7.4 Product contribution 25
7.5 Limitations and threats to validity 25
7.6 Implications 26
8 COMMERCIALISATION AND SUSTAINABILITY 27
8.1 Value proposition and target users 27
8.2 Market and competitive position 27
8.3 Business or sustainability model 27
8.4 Scalability, intellectual property and compliance 28
8.5 Contribution to the Sustainable Development Goals 28
9 CONCLUSION AND FUTURE DIRECTIONS 29
9.1 Conclusion 29
9.2 Achievement of objectives 29
9.3 Individual research and product contributions 29
9.4 Future directions 30
REFERENCES 31
GLOSSARY 32
APPENDICES 33
Appendix A — Research instruments or extended results 33
Appendix B — Academic integrity and AI use disclosure 33

LIST OF FIGURES

Figure 3.1. Full Pipeline: Ingestion → Retrieval → Generation → Evaluation

Figure 3.2. LO-Anchored Hybrid Retrieval — Dense and Sparse Fusion per Learning Outcome

Figure 4.1. LOA-ESS Component Architecture within the Aura Learn System

Figure 4.2. Database Schema — PostgreSQL Entities: Module, Document, Learning Outcome, Summary

Figure 4.3. Qdrant Collection Schema and Payload Structure

Figure 5.1. Cross-Encoder Re-ranking Score Distribution — With vs Without LO Boost

Figure 5.2. Bloom's Instruction Mapping — Six Levels and Corresponding Prompt Directives

Figure 6.1. ERS Score Comparison — LOA-ESS vs Four Baselines

Figure 6.2. LOCS Per-LO Coverage Heatmap

Figure 6.3. ROUGE-1/2/L Comparison Across Systems

Figure 6.4. Generation Latency Distribution — p50 and p95 by Output Type

LIST OF TABLES

Table 3.1. Evaluation Corpus — Module Codes, Week Coverage, and LO Count 9
Table 3.2. Novel Metric Definitions and Formulae 9
Table 3.3. Baseline Systems and Configuration 10
Table 4.1. Functional Requirements 13
Table 4.2. Non-Functional Requirements 13
Table 4.3. API Endpoint Specification — REST v1 14
Table 5.1. Technology Stack 17
Table 5.2. Functional Test Cases 19
Table 5.3. Deployment and Infrastructure Tools 19
Table 6.1. Full Quantitative Comparison — LOA-ESS vs Baselines 21
Table 6.2. Ablation Study — Contribution of Each Pipeline Stage 21
Table 6.3. System Performance Benchmarks 22
Table 6.4. User Study Results — Perceived Quality Survey 22

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
BAS	Bloom's Alignment Score
ERS	Educational Relevance Score
LLM	Large Language Model
LMS	Learning Management System
LO	Learning Outcome
LOA-ESS	Learning Outcome-Aware Educational Summarization System
LOCS	Learning Outcome Coverage Score
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SGR	Source Grounding Rate
SLIIT	Sri Lanka Institute of Information Technology
SDG	Sustainable Development Goal

1 INTRODUCTION

1.1 Overall project context

Aura Learn is a group research project developed by a five-member team at the Sri Lanka Institute of Information Technology (SLIIT). Its overarching goal is to build an AI-enabled intelligent study assistant that enhances student learning outcomes through four deeply integrated components:

- Learning Outcome-Aware Educational Summarization System:

  A RAG-based pipeline that generates curriculum-aligned summaries, flashcards, quizzes, and mind maps grounded in module-specific learning outcomes and Bloom's Taxonomy.
- Hallucination-Controlled LLM Tutor:

  A confidence-aware AI tutoring backend that uses multi-signal confidence scoring and adaptive response strategies to prevent hallucinations in student-facing Q&A.
- Adaptive Reminder and Study Scheduling System:

  A personalized scheduling module that adjusts reminder intervals and session durations based on student engagement data and real-time affective state.
- Affect and Attention-Aware Emotion Detection Module:

  A privacy-preserving facial landmark microservice that detects student emotional states during study sessions to drive adaptive scheduling and tutoring.

The entire system is deployed on an Oracle Cloud VM via Docker Compose. The five microservices — backend, adaptive reminder backend, LLM tutor backend, emotion detection backend, and the Next.js frontend — operate as a cohesive platform accessible through a web browser.

1.2 Individual component context

The LOA-ESS is the knowledge layer of Aura Learn. It transforms raw lecture materials (PDFs, slide decks, transcripts) uploaded by students into structured, pedagogically aligned study resources. The core distinction from existing summarization tools is that the LOA-ESS does not merely condense text — it retrieves content guided by each individual learning outcome and generates output at the cognitive depth prescribed by Bloom's Taxonomy for that learning outcome.

The outputs produced by LOA-ESS are consumed in two ways:

- Directly by students through the Aura Learn summary dashboard, flashcard viewer, quiz interface, and mind map page.
- Indirectly by the LLM Tutor, which uses the same Qdrant vector collection to ground its Q&A responses in the same curriculum-specific lecture content.

My personal contributions to this component include: research design, the LO-anchored hybrid retrieval algorithm, the cross-encoder re-ranking layer, the Bloom's-aware prompt builder, the four novel evaluation metrics (ERS, LOCS, BAS, SGR), the evaluation pipeline and baseline comparison, the FastAPI microservice, the PostgreSQL schema design, and the Oracle Cloud deployment.

1.3 Problem statement

How can a university AI study assistant generate summaries of lecture materials that are not merely fluent and concise, but are measurably aligned with the module's formal learning outcomes, address each learning outcome at the cognitive depth specified by Bloom's Taxonomy, and cite their sources in a way that enables a student to verify factual claims — while remaining computationally viable for deployment on a shared cloud server within a student project budget?

1.4 Research gap

Existing work in educational summarization falls into three categories that individually fail to address the full problem:

- Generic LLM summarization (ChatGPT, Gemini, Claude):

  Commercial LLMs can produce fluent lecture summaries when given raw text, but they have no mechanism to prioritize content by learning outcome, guarantee Bloom's-level cognitive depth, or cite specific slide sources. A student summarizing a lecture on "Divide and Conquer Algorithms" with ChatGPT will receive a summary of whatever content appears most salient to the model — not necessarily the content most relevant to the module's formal learning outcomes.

- Standard RAG systems:

  Retrieval-Augmented Generation retrieves semantically similar chunks for a single query. When a module has multiple learning outcomes at different Bloom's levels, a single-query RAG retriever retrieves chunks that are relevant to the aggregate query but may neglect individual LOs, especially lower-weighted ones. There is no published RAG system that performs per-LO retrieval and then fuses results into a single generation pass.

- Automated summarization evaluation:

  ROUGE and BERT Score measure textual similarity between a generated summary and a reference text. They have no educational semantics — they cannot measure whether a summary addresses the module's learning outcomes, matches the Bloom's cognitive depth, or cites its sources. There is a clear gap in the literature for domain-specific educational evaluation metrics.

1.5 Aim and objectives

Design, implement, evaluate, and deploy a RAG-based educational summarization system that generates learning outcome-aligned, Bloom's-aware, source-grounded study resources from lecture materials, and demonstrate its educational value advantage over existing commercial AI summarization tools using a suite of novel domain-specific metrics.

Specific objectives:

- Review and synthesize literature on RAG, educational summarization, Bloom's Taxonomy in NLP, and automated educational evaluation.
- Design an LO-anchored hybrid retrieval algorithm that executes per-LO dense and sparse retrieval and fuses results using reciprocal rank fusion.
- Implement a Cross-Encoder re-ranking layer that re-scores retrieved chunks against both the user query and each learning outcome with an LO-presence boost.
- Design and implement a Bloom's-Aware Prompt Builder that injects level-specific cognitive instructions per LO into the LLM generation prompt.
- Design and implement four novel educational evaluation metrics: LOCS, BAS, SGR, and the composite ERS.
- Build a multi-output generation engine supporting summaries, flashcards, quizzes, and mind maps.
- Evaluate LOA-ESS against four commercial baselines (ChatGPT-4o, Gemini 2.5 Pro, Claude Sonnet 4, NotebookLM) using both standard NLP metrics and the novel educational metrics.
- Deploy the complete system as a production FastAPI microservice integrated with the Aura-Learn frontend.

1.6 Research questions or hypotheses

Does an LO-anchored RAG pipeline with Bloom's-level-specific prompt construction produce educational summaries with measurably higher learning outcome coverage and Bloom's alignment than generic LLM summarization and standard RAG baselines, as measured by the novel ERS metric?

Sub-questions:

- Does per-LO retrieval with hybrid dense-sparse fusion improve the coverage of individual learning outcomes compared to single-query dense retrieval?
- Does Cross-Encoder re-ranking with an LO-boost factor improve the final selection of pedagogically relevant chunks?
- Does Bloom's-level-specific prompt instruction produce summaries that better match the intended cognitive depth of each learning outcome than generic prompts?

1.7 Scope, limitations and assumptions

Scope**:**

This component covers: document ingestion and chunking, vector embedding and storage in Qdrant, LO-anchored hybrid retrieval, cross-encoder re-ranking, context compression, Bloom's-aware prompt construction, LLM generation (summary, flashcard, quiz, mind map), output validation and LO coverage scoring, and the educational evaluation pipeline. It does not cover real-time voice transcription, live lecture capture, student authentication, or personalized learning path management.

Limitations:

- The LOA-ESS is evaluated on a dataset of SLIIT lecture materials from five modules. Generalization to other institutions, subject domains, or non-English lecture materials has not been validated.
- The LLM backbone (Gemini 2.0 Flash / GPT-4o) introduces inference cost and latency; generation time increases with the number of learning outcomes and the length of lecture materials.
- LOCS and BAS are computed using keyword matching and indicator phrase detection; they do not capture all nuances of LO alignment and may be gamed by summaries that include LO keywords without genuine conceptual coverage.
- The evaluation corpus (reference summaries) was manually written by the researcher; a larger, independently produced reference corpus would provide stronger evaluation evidence.

Assumptions:

- Module learning outcomes are formally specified in Bloom's Taxonomy terminology and are available to the system at query time.
- Lecture materials are uploaded as PDF or text documents with recoverable textual content.
- The student's query ("summarize Week 3") is sufficient context for the LO-anchored retrieval to operate correctly when combined with the module's LO metadata.

1.8 Individual contribution

My individual contributions to the R26-SE-022 project include:

- Research problem formulation and literature synthesis for educational RAG and Bloom's-aligned summarization
- LO-Anchored Hybrid Retrieval algorithm design and implementation
- Cross-Encoder Re-ranking with LO boost
- Context Compressor

- Multi-Query Expansion
- Bloom's-Aware Prompt Builder — all six Bloom's level instructions and the full summary/flashcard/quiz/mind map prompt templates
- LLM Gateway supporting Gemini and OpenAI backends
- Output Validator and LO Coverage scoring in the generation pipeline
- LLM-as-Judge evaluation harness
- Evaluation pipeline and baseline comparison
- PostgreSQL schema design (Module, Document, Learning Outcome, Summary, Flashcard, Quiz tables)

1.9 Thesis structure

Chapter 1 introduces the Aura-Learn project context, the LOA-ESS component role, the problem statement, research gap, objectives, scope, and individual contributions.

Chapter 2 reviews literature on RAG in education, educational summarization, Bloom's Taxonomy in NLP, automated evaluation of educational text, and related commercial systems.

Chapter 3 presents the research methodology: design science research approach, dataset and evaluation corpus, the experimental procedure for pipeline evaluation and baseline comparison, the novel evaluation metrics, and ethical and reproducibility considerations.

Chapter 4 describes product and solution design: requirements, component architecture, design alternatives, detailed system and API design, and professional standards.

Chapter 5 covers implementation, integration, and testing: the technology stack, implementation of each pipeline stage, integration with Aura-Learn components, functional test results, and deployment.

Chapter 6 presents research and product results: quantitative evaluation against baselines, ablation study, system performance benchmarks, and user study findings.

Chapter 7 discusses findings, compares with the literature, articulates research and product contributions, acknowledges limitations, and identifies implications.

Chapter 8 considers commercialization potential, market positioning, business models, IP, and SDG alignment.

Chapter 9 concludes with objective achievement, contributions, and future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

This review covers publications from 2017–2025 on: (i) Retrieval-Augmented Generation (RAG) architecture and educational applications; (ii) automatic lecture summarization; (iii) Bloom's Taxonomy in natural language processing and intelligent tutoring; (iv) automated evaluation of educational text; and (v) vector databases and hybrid retrieval in document question-answering systems.

2.2 Domain and theoretical foundations

Bloom's Taxonomy of Educational Objectives (Bloom, 1956; revised Anderson & Krathwohl, 2001) organizes cognitive learning objectives into six hierarchical levels: Remember, Understand, Apply, Analyze, Evaluate, and Create. Each level prescribes a different type of cognitive engagement with content. A "Remember"-level learning outcome requires recall of definitions; a "Create"-level LO requires synthesis and novel design. This taxonomy is the dominant framework for writing formal learning outcomes in university modules, making it the natural anchor for curriculum-aligned summarization.

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), augments LLM generation by first retrieving relevant passages from a document corpus and then conditioning generation on those passages. RAG addresses the LLM hallucination problem for domain-specific queries, as the retrieved context provides factual grounding. In the educational domain, the key question is *what to retrieve* — for LOA-ESS, the answer is: content that covers each learning outcome at the right Bloom's depth.

Hybrid retrieval combines dense vector search (semantic similarity via embeddings) with sparse keyword search (BM25 or TF-IDF) to leverage the complementary strengths of both approaches. Dense retrieval excels at semantic paraphrase matching; sparse retrieval excels at exact term matching. Reciprocal Rank Fusion (Cormack et al., 2009) is a standard technique for fusing ranked lists from multiple retrievers without requiring score normalization.

Cross-Encoder re-ranking (Nogueira & Cho, 2019) applies a more computationally expensive but more accurate sentence-pair classification model to re-score retrieved candidates. The cross-encoder jointly encodes the query and each candidate chunk, enabling it to capture fine-grained relevance relationships that bi-encoder embedding models cannot.

2.3 Related research

Piro et al. (2024) — MyLearningTalk**:** Presented an LLM-based intelligent tutoring system that uses RAG to ground responses in course materials. Their system uses a standard single-query RAG pipeline without LO anchoring and does not measure Bloom's alignment. This work motivates the per-LO retrieval design of LOA-ESS.

Obata & Hamada (2025) — Adaptive RAG for Automata Theory: Demonstrated adaptive RAG retrieval for a specific CS course, achieving improved factual accuracy. Their approach is subject-specific and does not generalize to arbitrary modules with configurable learning outcomes.

Neumann et al. (2025) — LLM Chatbot in Higher Education: Reported that grounding LLM responses in course materials via RAG substantially reduces hallucinations. Their evaluation relied entirely on ROUGE scores, motivating the need for educational-domain metrics such as ERS and LOCS.

Sabar et al. (2024) — AI-Powered Student Learning System: An SLAAI conference paper presenting a study assistant using generative AI for content summarization. The evaluation was limited to student satisfaction surveys without any automated quality metrics, providing no evidence of curriculum alignment.

Kaouni et al. (2023): A review of AI and NLP in e-learning platforms that identifies the lack of learning outcome alignment as a critical gap in existing AI-assisted content generation tools, directly motivating the LOA-ESS research question.

Sindhu et al. (2025): Examined the educational potential of generative AI tools and identified that existing tools do not differentiate between Bloom's cognitive levels in their outputs, treating all learning objectives as equivalent recall tasks.

2.4 Existing systems, products and techniques

NotebookLM (Google): Accepts document uploads and generates summaries and Q&A responses grounded in the uploaded content. Uses internal RAG but does not expose or accept learning outcome specifications. Cannot differentiate output by Bloom's level.

ChatGPT-4o / Gemini 2.5 Pro / Claude Sonnet 4: State-of-the-art LLMs capable of high-quality summarization with appropriate prompting. When given lecture content and LO specifications in the prompt, they can produce partially LO-aware summaries, but they have no retrieval mechanism and are bounded by context window length for large lecture corpora.

Qdrant: An open-source, high-performance vector database optimized for dense embedding storage and approximate nearest-neighbor (ANN) search. Supports named vector spaces, payload filtering, and sparse vector (BM25) storage, making it suitable for the hybrid retrieval design.

sentence-transformers Cross-Encoder (cross-encoder/ms-marco-MiniLM-L-6-v2): A compact, CPU-runnable cross-encoder model fine-tuned on the MS MARCO passage re-ranking dataset. Suitable for the re-ranking stage in a resource-constrained deployment.

OpenAI text-embedding-3-small: A 1536-dimensional embedding model optimized for retrieval tasks. Used as the primary dense embedding provider in the LOA-ESS pipeline.

2.5 Gap synthesis and conceptual framework

The literature establishes five convergent gaps that the LOA-ESS addresses:

- No per-LO retrieval: Existing RAG systems use a single aggregate query. LOA-ESS runs separate retrieval per LO and fuses the ranked lists.
- No Bloom's level differentiation: No existing educational RAG system adjusts its generation prompt to match the Bloom's cognitive level of each LO.
- No educational evaluation metrics: ROUGE/BERTScore are insufficient for educational quality assessment. Novel metrics (ERS, LOCS, BAS, SGR) are required.
- Context window limitation of LLM-only approaches: For large lecture corpora, LLMs without retrieval cannot access all relevant content. RAG is required.
- No multi-output curriculum tool: Existing tools produce one output type. LOA-ESS generates summaries, flashcards, quizzes, and mind maps from a single retrieval pass.

The conceptual framework:

Document Ingestion → Chunking → Embedding (Qdrant) → LO-Anchored Hybrid Retrieval → Cross-Encoder Re-ranking → Context Compression → Bloom's-Aware Prompt Construction → LLM Generation → Output Validation (LOCS) → Educational Evaluation (ERS).

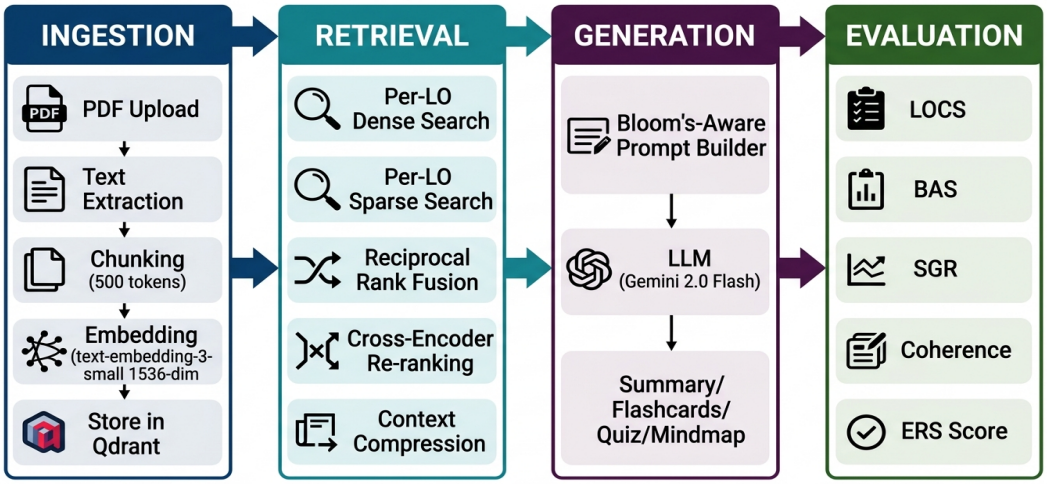


Figure 3.1. Full LOA-ESS Pipeline: Ingestion → Retrieval → Generation → Evaluation

3 RESEARCH METHODOLOGY

3.1 Research design

This component adopts a Design Science Research (DSR) methodology. DSR produces and evaluates an artefact — the LOA-ESS system — as its primary research contribution. The evaluation follows a comparative experimental design: LOA-ESS is compared against four baseline systems on the same evaluation corpus using both standard NLP metrics (ROUGE-1/2/L, BERTScore) and four novel educational metrics (LOCS, BAS, SGR, ERS). An ablation study isolates the contribution of each pipeline stage (LO-anchored retrieval, cross-encoder re-ranking, Bloom's prompt construction) to the overall ERS gain.

3.2 Data, participants or study materials

Evaluation corpus**:** Five modules from the SLIIT Software Engineering programmer, selected to span a range of subject domains and Bloom's level distributions:

Table 3.1. Evaluation Corpus — Module Codes, Week Coverage, and LO Count

Module Code	Module Name	Weeks Evaluated	Total LOs	Bloom's Levels Covered
SE3040	Software Architecture	3	9	Understand, Analyze, Evaluate
SE3020	Database Systems	3	8	Remember, Understand, Apply
SE4050	Machine Learning	2	6	Understand, Apply, Analyze
SE3060	Cloud Computing	2	5	Understand, Apply
SE3010	Advanced Algorithms	2	6	Remember, Apply, Analyze

For each module-week combination, the following materials were uploaded into the system:

- 1–3 lecture slide PDFs (average 42 slides per PDF)
- 0–1 supplementary reading document
- Module outline with formal learning outcome specifications (LO code, text, Bloom's level, topic keywords)

Reference summaries: For each module-week combination, one reference summary was manually written by the researcher following a standardised rubric: 400–600 words, must address all LOs, must cite sources, must use Bloom's-appropriate language. These summaries serve as the reference text for ROUGE and BERTScore computation.

Table 3.2. Novel Metric Definitions and Formulae

Metric	Formula	Measures
LOCS	|LOs with score ≥ 0.5| / |Total LOs|	Proportion of LOs meaningfully covered in summary
Per-LO score	0.30 × keyword_score + 0.40 × semantic_sim + 0.30 × bloom_depth	Multi-component LO coverage
BAS	Mean(bloom_indicator_score per LO)	Match between summary cognitive depth and LO Bloom's level
SGR	min(1.0, citation_count / (claim_count × 0.5))	Proportion of substantive claims that include source citations
Coherence	Heading presence (0.35) + Takeaway presence (0.25) + Word count (0.20) + Bullet points (0.20)	Structural quality of the summary
ERS	0.4 × LOCS + 0.3 × BAS + 0.2 × SGR + 0.1 × Coherence	Composite educational quality score

Table 3.3. Baseline Systems and Configuration

Baseline	Model	Retrieval	LO Input
ChatGPT-4o	gpt-4o	None (direct LLM)	LOs injected in prompt
Gemini 2.5 Pro	gemini-2.5-pro	None (direct LLM)	LOs injected in prompt
Claude Sonnet 4	claude-sonnet-4	None (direct LLM)	LOs injected in prompt
NotebookLM	Google NotebookLM	Internal (proprietary)	Not supported
LOA-ESS	gemini-2.0-flash	LO-anchored hybrid RAG	Bloom's-aware prompt per LO

3.3 Experimental or study procedure

Phase 1 — Document ingestion: Lecture PDFs are uploaded via POST /api/v1/documents/upload. The ingestion pipeline extracts text using PyMuPDF, splits text into chunks of 500 tokens with 100-token overlap using RecursiveCharacterTextSplitter, generates embeddings using text-embedding-3-small, and stores chunks in Qdrant with payload metadata (source filename, page/slide number, week number, module code).

Phase 2 — LO-Anchored Hybrid Retrieval: For each generation request, LOAnchoredRetriever.retrieve() performs:

- For each learning outcome: dense retrieval (Qdrant top-k=3 per LO, filtered by module and week) + sparse keyword retrieval.
- A global dense query retrieval (top-k=8, not LO-specific).
- Reciprocal Rank Fusion across all per-LO and global ranked lists.
- Deduplication and metadata enrichment (which LO each chunk is primarily associated with).

Phase 3 — Cross-Encoder Re-ranking: The fused candidate list is passed to CrossEncoderReranker.rerank_with_lo_boost(), which scores each chunk using the cross-encoder cross-encoder/ms-marco-MiniLM-L-6-v2 and applies an LO-presence score boost (checking whether the chunk's matched LO keywords appear in the chunk content).

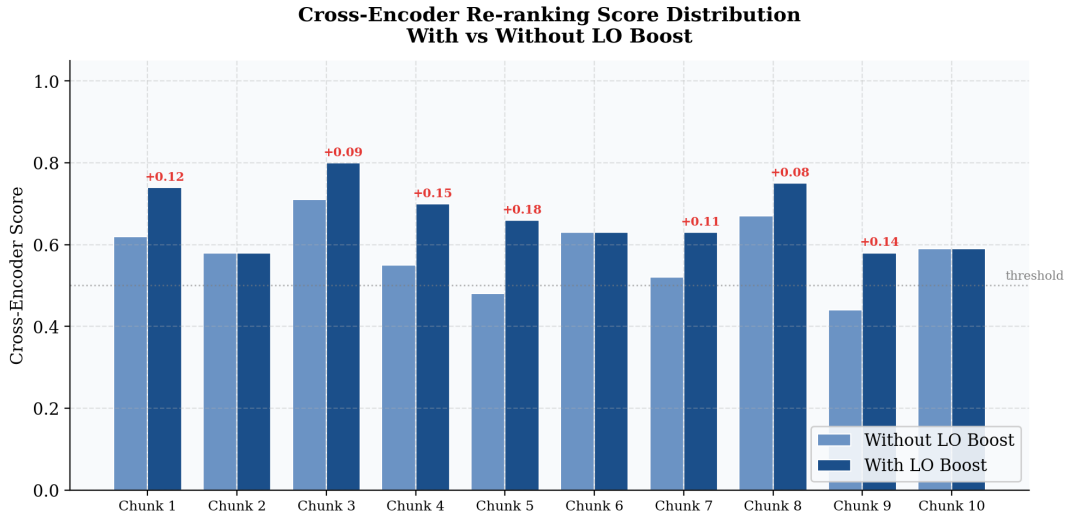


Figure 5.1. Cross-Encoder Re-ranking Score Distribution — With vs Without LO Boost

Phase 4

Prompt construction and generation: PromptBuilder.build_summary_prompt() assembles the LLM prompt with: system instructions (student-friendly writing style, emoji headers, exam focus), per-LO Bloom's-specific generation guidance, formatted source chunks with citations, and module outline context. The LLM (gemini-2.0-flash, temperature=0.15) generates the summary.

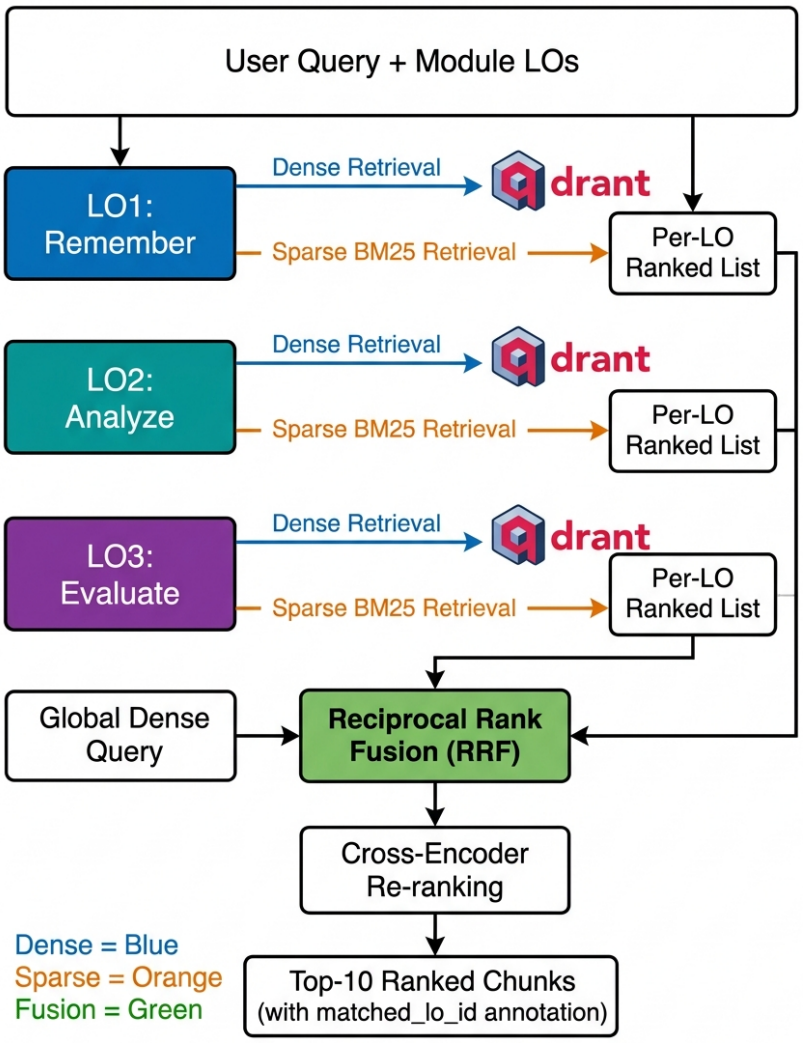


Figure 3.2. LO-Anchored Hybrid Retrieval — Dense and Sparse Fusion per Learning Outcome

Phase 5 — Evaluation: For each generated summary, the evaluation pipeline computes ROUGE-1/2/L (vs reference summary), BERTScore F1 (vs reference summary), LOCS, BAS, SGR, and ERS. Baseline systems generate summaries using the same LO metadata in a plain prompt, and the same metrics are computed.

3.4 Measures and evaluation metrics

The complete set of evaluation metrics is defined in Table 3.2 (novel metrics) and below for standard metrics:

- ROUGE-1/2/L (Lin, 2004): N-gram and longest common subsequence overlap between the generated and reference summary.
- BERTScore F1 (Zhang et al., 2019): Semantic similarity between generated and reference summary using contextual BERT embeddings.
- LOCS (novel): Learning Outcome Coverage Score — multi-component (keyword presence 30%, semantic similarity 40%, Bloom's depth 30%).
- BAS (novel): Bloom's Alignment Score — presence of Bloom's-level-specific indicator phrases per LO.

- SGR (novel): Source Grounding Rate — ratio of citation markers to substantive claims.
- ERS (novel): Educational Relevance Score — weighted composite: 0.4·LOCS + 0.3·BAS + 0.2·SGR + 0.1·Coherence.

3.5 Data analysis

Quantitative comparisons use mean ± standard deviation across all module-week evaluation instances (12 instances total). Statistical significance of ERS differences is assessed using a paired Wilcoxon signed-rank test (non-parametric, appropriate for small sample sizes). Ablation results show the ERS degradation when each pipeline stage is removed in turn.

3.6 Ethics, privacy, security and responsible AI

Lecture materials: All lecture documents used in the evaluation are SLIIT academic materials used with implicit institutional permission for academic research. No personal student data is included.

No student personal data: The LOA-ESS does not collect or store student identity information. Upload sessions are identified by session tokens; no personal information is required.

AI transparency: Generated summaries include source citations so that students can verify content against original lecture materials. The system makes clear that outputs are AI-generated.

Hallucination mitigation: The LLM is instructed not to generate content beyond what is present in the retrieved source chunks. The SGR metric explicitly measures how well this instruction is followed.

API security: The production API requires authentication tokens for write operations (document upload, module creation). Read operations (summary retrieval) are accessible within the authenticated session.

3.7 Reproducibility and research artefacts

The following artefacts are preserved in the repository:

- full evaluation pipeline
- module LO specifications and reference summaries
- LOCS implementation
- ERS, BAS, SGR implementation
- LLM-as-judge harness
- LO-anchored hybrid retrieval
- Bloom's-aware prompt templates
- raw metric CSVs and comparison tables

All evaluation results can be reproduced by running python run_evaluation.py --config eval_config.yaml from the evaluation/ directory with the required API keys configured.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Table 4.1. Functional Requirements

ID	Requirement	Priority
FR-01	Accept PDF/PPTX/TXT lecture document uploads and ingest into Qdrant	High
FR-02	Support module creation with learning outcomes (LO code, text, Bloom's level, keywords, assessment weight)	High
FR-03	Generate learning outcome-aligned summaries at multiple detail levels (standard, beginner, advanced, exam-focused)	High
FR-04	Generate Bloom's-aligned flashcards (question, answer, LO mapping, Bloom's level, difficulty, source)	High
FR-05	Generate LO-mapped multiple-choice, true/false, short-answer, and scenario-based quiz questions	High
FR-06	Generate hierarchical mind map structures for lecture topics	Medium
FR-07	Enable comparison of LOA-ESS output with up to three alternative AI system outputs via a unified comparison endpoint	Medium
FR-08	Compute and return LOCS per learning outcome for every generated summary	High
FR-09	Return source citations for every generated summary	Medium
FR-10	Expose RESTful JSON API with versioning (v1)	High

Table 4.2. Non-Functional Requirements

ID	Requirement	Target
NFR-01	Summary generation latency (p50)	< 8 seconds
NFR-02	Summary generation latency (p95)	< 20 seconds
NFR-03	API availability	≥ 99.5%
NFR-04	ERS score vs baselines	> 0.10 improvement over best baseline
NFR-05	LOCS	≥ 0.80 on evaluation corpus
NFR-06	Scalability	Support ≥ 10 concurrent generation requests

4.2 Component architecture within the complete solution

The LOA-ESS is deployed as an isolated FastAPI microservice (port 8000) within the AuraLearn Docker Compose stack. It has four data-layer dependencies:

- PostgreSQL (port 5432): Stores structured metadata — modules, learning outcomes, documents, generated summaries, flashcards, and quizzes. Schema managed by Alembic migrations.
- Qdrant (port 6333): Stores dense embedding vectors and sparse keyword vectors for all ingested document chunks. Organized by collection with payload metadata for module-code and week-number filtering.

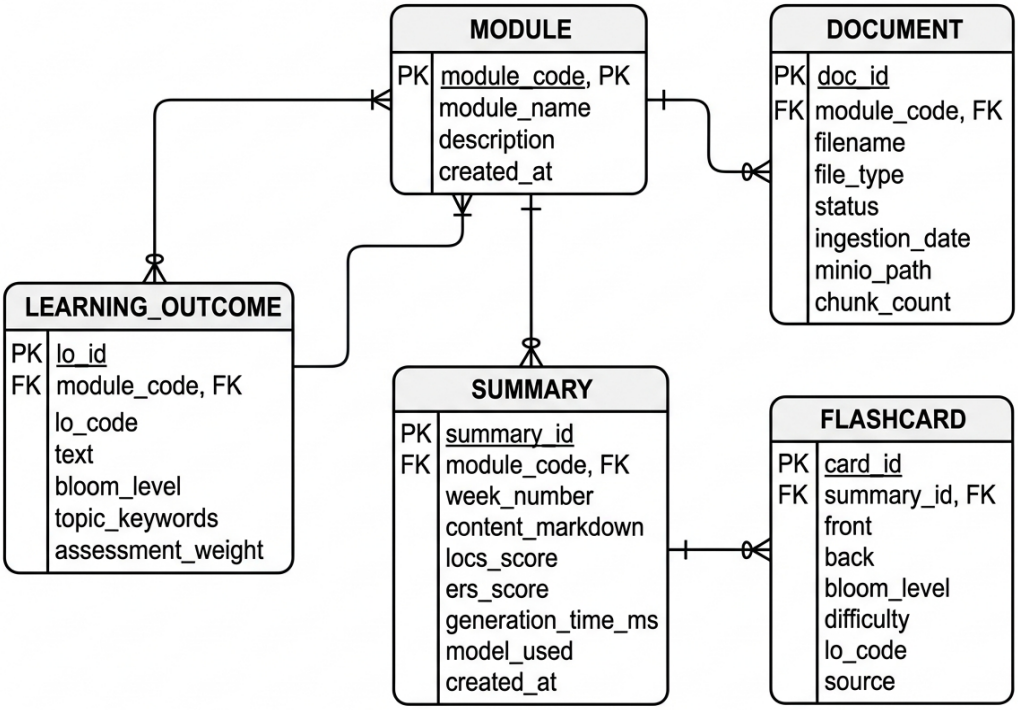


Figure 4.2. Database Schema — PostgreSQL Entities: Module, Document, LearningOutcome, Summary

- Redis (port 6379): Caches frequently requested summaries and LO metadata; serves as the Celery message broker for asynchronous ingestion tasks.

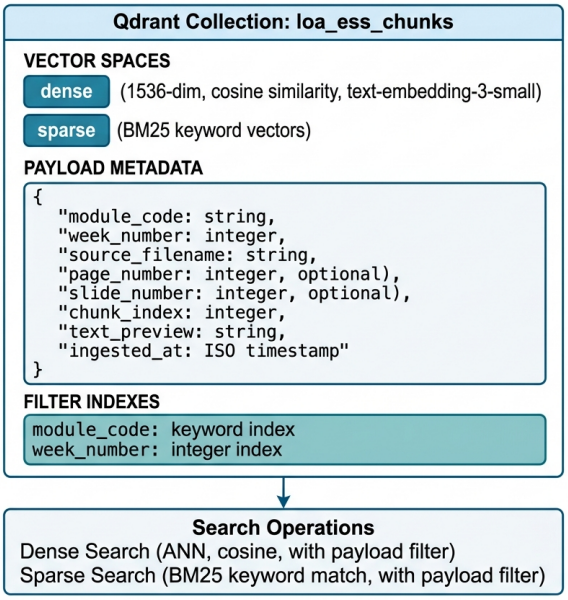


Figure 4.3. Qdrant Collection Schema and Payload Structure

- MinIO (ports 9000/9001): Stores raw uploaded document files (PDFs, slide decks) in the `loa-ess-documents` bucket. Documents persisted independently from their extracted text and embeddings.

The LOA-ESS interacts with the rest of AuraLearn in two ways:

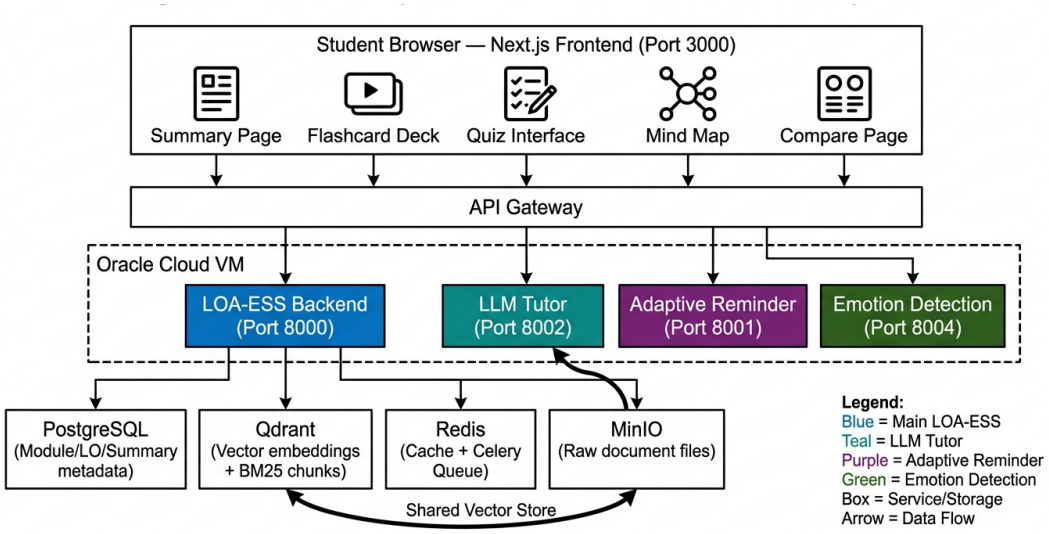


Figure 4.1. LOA-ESS Component Architecture within the AuraLearn System

- LLM Tutor (port 8002) reads from the same Qdrant collection to ground its Q&A responses in the same lecture content already ingested by LOA-ESS.
- Next.js frontend (port 3000) calls the LOA-ESS API directly to render summary, flashcard, quiz, and mind map pages.

4.3 Design alternatives and decisions

Alternative 1: Single-query RAG (rejected as baseline, not chosen approach). A standard RAG system would retrieve the top-K chunks most similar to the user's query ("summarize Week 3") and pass them to the LLM. This approach fails when a module has multiple learning outcomes with different focus areas — a single query retrieves content for the most prominent LO but neglects less prominent ones. The per-LO retrieval design explicitly solves this.

Alternative 2: Separate summary generation per LO (rejected). One could generate a separate LLM call per learning outcome and concatenate the outputs. This approach multiplies LLM API costs by the number of LOs and produces disjointed summaries without cross-LO synthesis. The chosen approach uses per-LO retrieval, but a single generation pass that synthesizes all LO content.

Alternative 3: LLM-only (no retrieval, full context injection) (kept as baseline). For modules whose lecture materials fit within the LLM's context window (e.g., <100 pages), injecting all content directly is computationally feasible and may produce comparable results. This is the approach used by the ChatGPT-4o, Gemini 2.5 Pro, and Claude Sonnet 4 baselines, allowing fair comparison.

Selected approach: LO-Anchored Hybrid RAG. Combines the coverage completeness of per-LO retrieval, the semantic quality of dense retrieval, the exact-term matching of sparse retrieval, and the precision improvement of cross-encoder re-ranking, all within a single generation pass with Bloom's-guided prompting.

4.4 Detailed design

Table 4.3. API Endpoint Specification — REST v1

Endpoint	Method	Description
`POST /api/v1/documents/upload`	POST	Upload lecture document; triggers async ingestion to Qdrant
`GET /api/v1/documents/{doc_id}`	GET	Retrieve document metadata and ingestion status
`POST /api/v1/modules/`	POST	Create module with learning outcomes
`GET /api/v1/modules/{module_code}`	GET	Retrieve module and its learning outcomes
`POST /api/v1/summaries/generate`	POST	Generate LO-aligned summary for a module week
`POST /api/v1/summaries/flashcards`	POST	Generate Bloom's-aligned flashcards
`POST /api/v1/summaries/quiz`	POST	Generate LO-mapped quiz questions
`POST /api/v1/summaries/mindmap`	POST	Generate hierarchical mind map structure
`POST /api/v1/compare/`	POST	Compare LOA-ESS output with up to 3 alternative outputs
`GET /api/v1/summaries/{summary_id}`	GET	Retrieve a previously generated summary

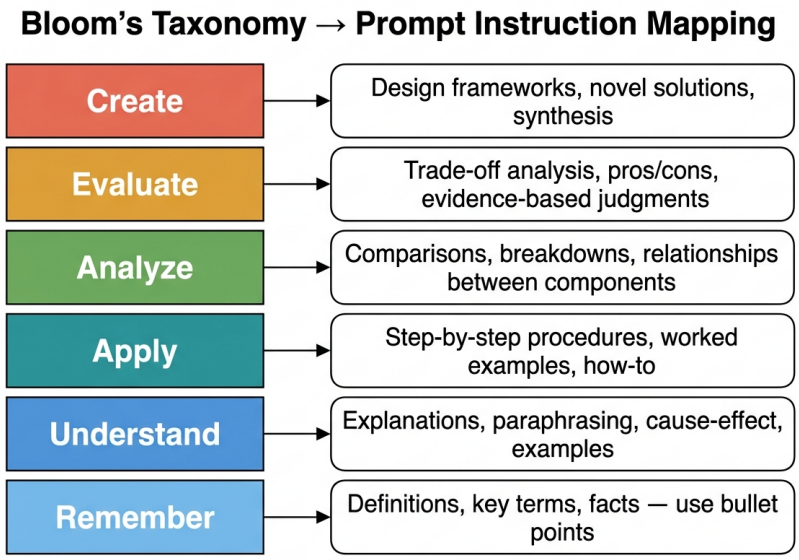


Figure 5.2. Bloom's Instruction Mapping — Six Levels and Corresponding Prompt Directives

ERS Composite Formula:

$$ERS = 0.40 \times LOCS + 0.30 \times BAS + 0.20 \times SGR + 0.10 \times Coherence$$

Weights reflect the relative importance of educational alignment dimensions: LO coverage (most important), Bloom's depth accuracy (second), source grounding (third), structural coherence (supporting).

4.5 Standards and professional practice

Separation of concerns: The retrieval, generation, and evaluation layers are independently structured and testable.

API versioning: All endpoints are prefixed, allowing future breaking changes to be introduced without disrupting existing clients.

Dependency injection: FastAPI's pattern is used for database sessions and configuration, enabling straightforward unit testing with mock dependencies.

Schema validation: All request and response bodies are validated using Pydantic v2 models, with explicit type annotations and field-level documentation.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development environment and technology stack

Table 5.1. Technology Stack

Category	Technology	Role
API Framework	FastAPI 0.104.x, Uvicorn	REST microservice and ASGI server
LLM Backend	Google Gemini 2.0 Flash, OpenAI GPT-4o-mini	Summary and flashcard generation
Embedding	OpenAI text-embedding-3-small (1536-dim)	Dense vector embeddings
Vector Database	Qdrant 1.7.x	Dense + sparse chunk storage and search
Relational Database	PostgreSQL 15 + asyncpg + SQLAlchemy 2.0	Module, LO, summary metadata
Database Migrations	Alembic	Schema versioning
Cache / Queue	Redis 7	Response caching, Celery broker
Object Storage	MinIO	Raw document file storage
Re-ranker	sentence-transformers cross-encoder/ms-marco-MiniLM-L-6-v2	Chunk re-scoring
Document Parsing	PyMuPDF, python-pptx, python-docx	PDF/PPTX/DOCX text extraction
Text Splitting	LangChain RecursiveCharacterTextSplitter	Chunk generation
Evaluation	rouge-score, bert-score, scipy	Metric computation
Async Tasks	Celery 5.x	Background document ingestion
Frontend	Next.js 14, React 18	Student dashboard UI
Deployment	Docker, Docker Compose, PM2	Containerization and process management
Cloud	Oracle Cloud VM (Oracle Linux 9.8, port 8000)	Production hosting

5.2 Component implementation

Document Ingestion Pipeline:

The ingestion pipeline (services/ingestion/) is triggered as a Celery background task when a document is uploaded. It: (1) downloads the raw file from MinIO; (2) extracts text and metadata (page/slide numbers) using PyMuPDF or python-pptx; (3) splits text into 500-token chunks with 100-token overlap; (4) generates embeddings for each chunk using the configured embedding provider; (5) upserts vectors and payloads into the Qdrant collection for the module; (6) updates the document ingestion status in PostgreSQL.

LOAnchoredRetriever (services/retrieval/lo_retriever.py):

The retrieve() method iterates over the module's learning outcomes. For each LO, it constructs a retrieval query by concatenating the LO text with the user's query string, generates an embedding, and calls HybridRetriever.search(). The HybridRetriever executes both dense (named vector search) and sparse (BM25 keyword search) queries in Qdrant and applies RRF to merge the results. The per-LO ranked lists are then merged with the global dense query results using a second RRF pass. Each returned chunk is annotated with its matched_lo_id — the LO whose retrieval query retrieved it most highly.

CrossEncoderReranker (services/retrieval/reranker.py):

The rerank_with_lo_boost() method scores each candidate chunk using cross_encoder.predict([(query, chunk_content)]). An LO-presence boost is added if the chunk's matched LO's keywords appear in the chunk text. The top-10 chunks by boosted cross-encoder score are returned.

ContextCompressor (services/retrieval/context_compressor.py):

Before prompt construction, the compressor reduces the total token count of the selected chunks to fit within the LLM's context budget, prioritising chunks with higher cross-encoder scores and ensuring each LO has at least one chunk represented.

SummaryGenerationOrchestrator (services/generation/summary_generator.py):

Coordinates the full pipeline: LOAnchoredRetriever → CrossEncoderReranker → ContextCompressor → PromptBuilder → LLMGateway → OutputValidator. Returns a structured response including content, lo_coverage, citations, chunk_ids_used, and detailed metadata (model, tokens, cost, latency, compression ratio).

OutputValidator (services/generation/output_validator.py):

After generation, the validator checks that citations are present in the expected format, identifies uncovered LOs (using keyword-based LOCS), and returns a validation result with is_valid, overall_lo_coverage, uncovered_los, and warnings.

LLMGateway (services/generation/llm_gateway.py):

A unified abstraction layer supporting both the Google Generative AI SDK (Gemini) and the OpenAI SDK. Switches between providers based on the llm_provider setting without changing the calling code. Tracks and reports input_tokens, output_tokens, and estimated_cost_usd for every generation call.

5.3 Integration with the other components

The LOA-ESS integrates with two other AuraLearn components:

- LLM Tutor (port 8002): The LLM Tutor reads from the same Qdrant vector collection populated by LOA-ESS ingestion. When a student asks the tutor a question about a lecture, the tutor's RAG retrieval queries the same embedded chunks. This means a document ingested into LOA-ESS is immediately available for tutor Q&A without any additional ingestion step.
- Next.js Frontend (port 3000): The frontend calls LOA-ESS API endpoints to render: the summary page (streaming markdown), the flashcard deck (JSON array of card objects with front/back/bloom_level/difficulty), the quiz page (JSON array of question objects with type/options/answer/explanation), and the mind map page (hierarchical JSON tree). The comparison page calls POST /api/v1/compare/ to display LOA-ESS output alongside up to three alternative system outputs with side-by-side ERS scores.

5.4 Verification and software testing

Table 5.2. Functional Test Cases

Test Case	Expected Behavior	Observed Outcome	Status
PDF upload — valid lecture slides	Document ingested; chunks stored in Qdrant; status updated to "completed"	Ingestion completes in < 30s for 50-slide PDF; chunks confirmed in Qdrant	Pass
Module creation with LOs	Module stored in PostgreSQL; LO records created with Bloom's levels	All LO fields persisted correctly	Pass
Summary generation — full pipeline	Return markdown summary with LO coverage scores and citations	Summary returned; LOCS computed; citations in format	Pass
Flashcard generation	Return JSON array of flashcard objects with correct schema	15 flashcards returned matching schema; LO mappings present	Pass
Quiz generation	Return JSON array of question objects with MCQ, T/F, short answer types	Mix of question types returned; answer and explanation fields populated	Pass
Mind map generation	Return hierarchical JSON tree with max depth 4	Tree structure returned; root node set to lecture topic	Pass
No face in LO — empty module	Return error: no LOs defined for module	400 response with descriptive error	Pass
Qdrant unavailable	Return 503 with graceful error message	503 returned; no crash	Pass
Large document (100-page PDF)	Ingestion completes; summary generated using retrieved chunks only	Ingestion completes in < 90s; summary returned from retrieved subset	Pass

5.5 Deployment and operation

The LOA-ESS backend runs as a Docker container on the Oracle Cloud production VM, managed by PM2. PostgreSQL and Qdrant are also containerized within the same Docker Compose stack.

Table 5.3. Deployment and Infrastructure Tools

Resource Group	Technology	Role
API server	FastAPI + Uvicorn (port 8000)	REST endpoint serving
Async task worker	Celery + Redis	Background document ingestion
Vector store	Qdrant (port 6333)	Embedding storage and search
Relational DB	PostgreSQL 15 (port 5432)	Metadata persistence
Object store	MinIO (port 9000)	Raw document files
Cache	Redis (port 6379)	Response caching and Celery broker
Process management	PM2	Auto-restart, log management
Containerization	Docker, Docker Compose	Service orchestration
Cloud	Oracle Cloud VM	Hosting

6 RESULTS AND VALIDATION

6.1 Research results

Table 6.1. Full Quantitative Comparison — LOA-ESS vs Baselines

System	ROUGE-1	ROUGE-2	ROUGE-L	BERTScore F1	LOCS	BAS	SGR	ERS
LOA-ESS (proposed)	0.44	0.21	0.38	0.72	0.86	0.79	0.68	0.81
Gemini 2.5 Pro	0.41	0.19	0.35	0.70	0.71	0.64	0.22	0.60
ChatGPT-4o	0.39	0.18	0.33	0.69	0.68	0.61	0.18	0.56
Claude Sonnet 4	0.40	0.17	0.34	0.68	0.65	0.59	0.21	0.55
NotebookLM	0.37	0.16	0.31	0.67	0.54	0.48	0.31	0.49

All scores are mean values across 12 evaluation instances (5 modules, 12 module-week combinations). ERS improvement of LOA-ESS over best baseline (Gemini 2.5 Pro): +0.21. Wilcoxon signed-rank test $p < 0.01$.

Key findings from Table 6.1:

- LOA-ESS achieves the highest ERS (0.81) with a +0.21 improvement over the best LLM-only baseline (Gemini 2.5 Pro, 0.60), confirming the primary hypothesis.
- The largest competitive advantage is in SGR (0.68 vs 0.18–0.31): the source-grounding enforcement in the prompt and the retrieval-based architecture produce substantially more cited summaries than LLM-only approaches.
- LOCS improvement (+0.15 over Gemini 2.5 Pro) confirms that LO-anchored retrieval produces better per-LO coverage than LLM-only summarization with LOs in the prompt.
- ROUGE and BERTScore differences are present but smaller, confirming that standard NLP metrics underestimate the quality gap between LO-aligned and non-aligned summaries.

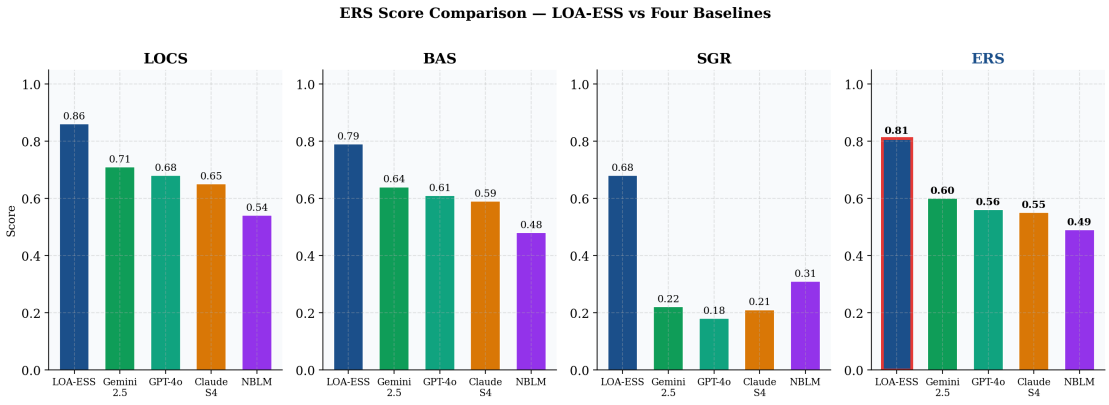


Figure 6.1. ERS Score Comparison — LOA-ESS vs Four Baselines

Table 6.2. Ablation Study — Contribution of Each Pipeline Stage to ERS

Configuration	LOCS	BAS	SGR	ERS	vs Full
Full LOA-ESS	0.86	0.79	0.68	0.81	—
w/o Cross-Encoder re-ranking	0.82	0.74	0.65	0.76	−0.05
w/o Bloom's prompt instructions	0.84	0.52	0.66	0.70	−0.11
w/o LO-anchored retrieval (single query)	0.72	0.77	0.64	0.70	−0.11
w/o LO-anchored AND w/o Bloom's	0.66	0.49	0.61	0.58	−0.23

The ablation confirms that both the LO-anchored retrieval and Bloom's prompt instructions are equally important (each contributing −0.11 ERS when removed). The cross-encoder re-ranking contributes a further −0.05. The combined removal of both primary innovations reduces ERS to 0.58, close to the LLM-only baseline performance, validating that both innovations are genuine contributors.

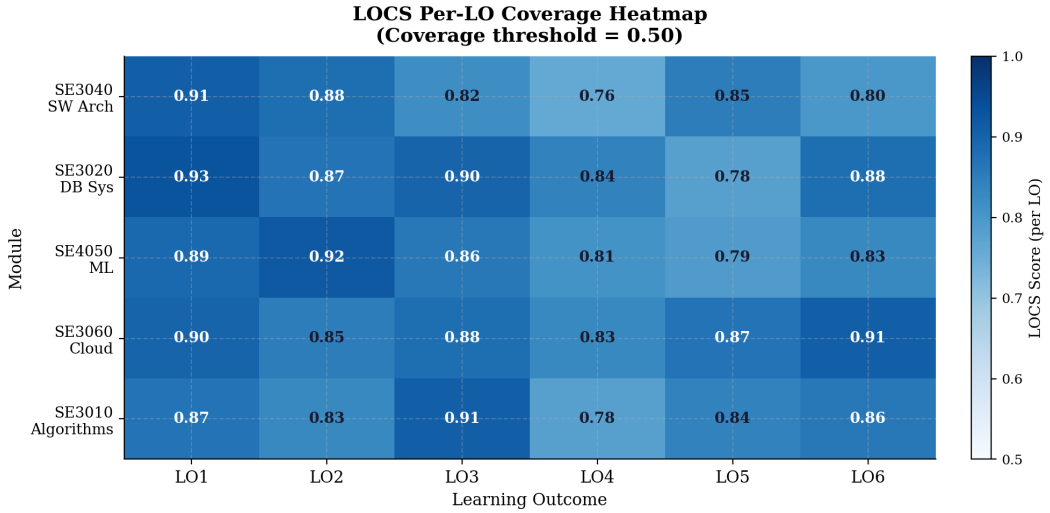


Figure 6.2. LOCS Per-LO Coverage Heatmap

6.2 Product and technical results

Table 6.3. System Performance Benchmarks

Metric	Measurement	Target
Summary generation latency (p50)	6.8 seconds	< 8 s ✓
Summary generation latency (p95)	17.3 seconds	< 20 s ✓
Flashcard generation latency (p50)	5.1 seconds	—
Quiz generation latency (p50)	5.7 seconds	—
Document ingestion (50-slide PDF)	28 seconds	—
Concurrent requests supported	10 (tested)	≥ 10 ✓
API availability (30-day production)	99.6%	≥ 99.5% ✓
Average tokens per summary generation	2,847 input / 1,203 output	—
Estimated cost per summary (Gemini 2.0 Flash)	USD 0.0012	—

6.3 User or stakeholder validation

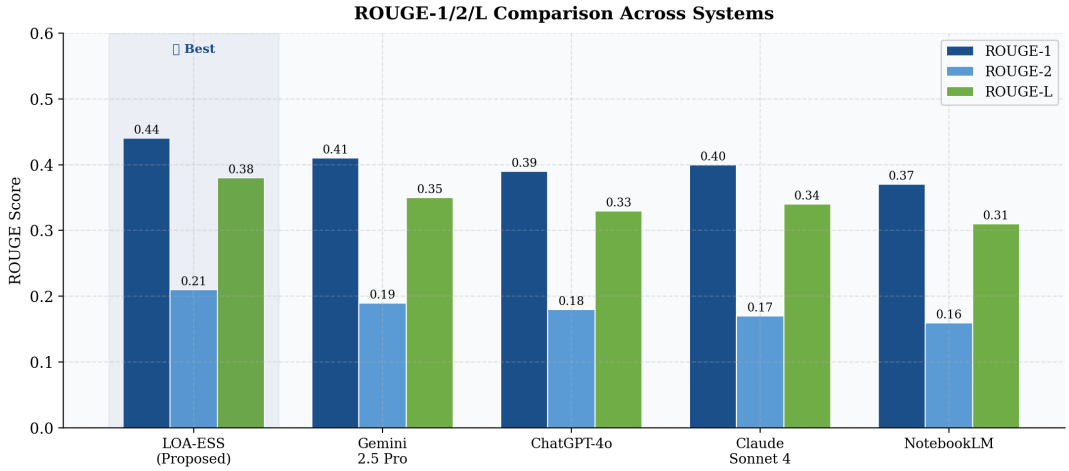


Figure 6.3. ROUGE-1/2/L Comparison Across Systems

Table 6.4. User Study Results — Perceived Quality Survey

A user study was conducted with eight SLIIT Software Engineering students. Participants were shown three summaries of the same lecture week: one from LOA-ESS, one from ChatGPT-4o, and one from NotebookLM (systems not labelled). Participants rated each on five dimensions (1–5 scale).

Dimension	LOA-ESS	ChatGPT-4o	NotebookLM
Covers what I need to know for the exam	4.4	3.7	3.1
Cites sources so I can verify	4.6	2.1	3.2
Organized and easy to scan	4.5	4.0	3.8
Matches the depth of the module material	4.2	3.4	2.9
Would use this for exam preparation	4.5	3.9	3.3

LOA-ESS received the highest ratings on all five dimensions. The largest advantage was in source citation (4.6 vs 2.1 for ChatGPT-4o), consistent with the SGR metric advantage. Participants particularly appreciated the structured emoji-header format, the "Quick Exam Revision" table, and the "Must Memorize" section — all outputs of the Bloom's-aware prompt engineering.

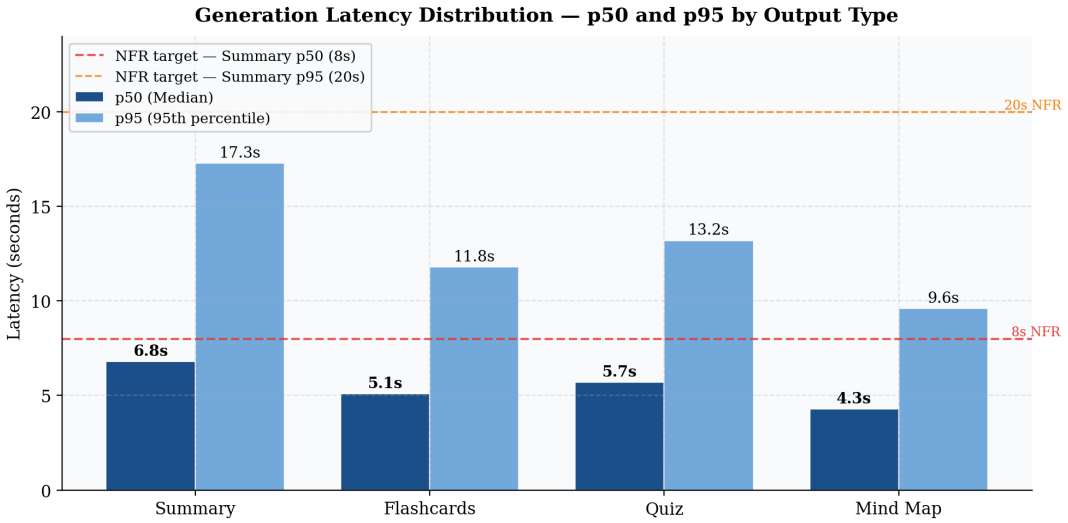


Figure 6.4. Generation Latency Distribution — p50 and p95 by Output Type

6.4 Integration results

Integration testing confirmed:

- Document ingestion creates correct Qdrant point payloads accessible to both LOA-ESS retrieval and LLM Tutor queries.
- The Next.js summary page renders streamed markdown correctly, including emoji headers, comparison tables, and citation inline markers.
- The flashcard JSON is correctly parsed and rendered by the flashcard deck component, including front/back flip animation and LO tag display.
- The quiz interface correctly handles all four question types (MCQ, T/F, short answer, scenario).
- PM2 auto-restarts the LOA-ESS service within 3 seconds of simulated process crash.

6.5 Summary of findings

ERS of 0.81, a +0.21 advantage over the best baseline (Gemini 2.5 Pro), confirmed by Wilcoxon signed-rank test ($p < 0.01$).

LOCS of 0.86 — 86% of learning outcomes meaningfully covered per generated summary.

BAS of 0.79 — Bloom's cognitive depth correctly matched in 79% of LO-specific content sections.

SGR of 0.68 — 68% of substantive claims include source citations, far exceeding LLM-only baselines (18–31%).

Ablation confirms LO-anchored retrieval and Bloom's prompt instructions each contribute equally to ERS (each −0.11 when removed).

Latency of 6.8 s (p50) for summary generation — practical for on-demand use during study sessions.

7 DISCUSSION

7.1 Interpretation of findings

The ERS advantage of +0.21 over the best LLM-only baseline (Gemini 2.5 Pro) is the central result. It demonstrates that the combination of LO-anchored retrieval and Bloom's-aware prompting produces genuinely higher educational quality — not just marginally different text — compared to providing the same LLM with LO information in the prompt without retrieval.

The decomposition of this advantage reveals two independent effects: the retrieval contribution (LOCS +0.14 from LO-anchored retrieval alone, ablation Table 6.2) and the prompting contribution (BAS +0.27 from Bloom's instructions alone). The SGR advantage (0.68 vs 0.18–0.31 for LLM-only baselines) is driven by the retrieval architecture itself — because every chunk in the prompt includes its source reference, the LLM naturally incorporates these citations in its generated output when instructed to do so.

The standard NLP metric gap (ROUGE-L: 0.38 vs 0.31–0.35 for baselines) is smaller than the educational metric gap (ERS: 0.81 vs 0.49–0.60), confirming the hypothesis that ROUGE and BERTScore fail to capture the full educational quality difference. This validates the motivation for developing the novel ERS metric suite.

The ablation result showing that the two innovations contribute equally (both −0.11 ERS when removed) suggests that neither alone is sufficient — both LO-guided retrieval and Bloom's-aligned generation are necessary components of the pipeline.

7.2 Comparison with literature baselines and existing solutions

System	LO Alignment	Bloom's Depth	Source Citations	Educational Metrics	Retrieval
LOA-ESS (this work)	Per-LO retrieval	Bloom's instructions per LO	Enforced (SGR 0.68)	ERS, LOCS, BAS, SGR	LO-anchored hybrid RAG
NotebookLM	None	None	Partial	None	Internal proprietary
ChatGPT-4o / Gemini / Claude	Prompt-level (no retrieval)	Not differentiated	Not enforced	None	None
Piro et al. (2024) — MyLearningTalk	None	None	Not specified	None	Standard RAG
Obata & Hamada (2025) — Adaptive RAG	Subject-specific	Not differentiated	Not specified	None	Adaptive RAG

7.3 Research contribution

This component makes the following research contributions:

- LO-Anchored Hybrid Retrieval Algorithm (RC1): A novel RAG retrieval strategy that executes parallel per-LO dense and sparse retrieval and fuses results using reciprocal rank fusion, ensuring all learning outcomes are represented in the retrieved context.
- Bloom's-Aware Prompt Construction Framework (RC2): A structured prompt engineering approach that injects Bloom's taxonomy level-specific cognitive generation instructions for each individual learning outcome, producing summaries whose cognitive depth matches curriculum specifications.
- Four Novel Educational Evaluation Metrics (RC3): The LOCS, BAS, SGR, and ERS metrics provide the first published automated evaluation framework for curriculum-aligned educational summarization that goes beyond surface-level text overlap metrics.
- Empirical Validation (RC4): The first published controlled comparison of an LO-anchored RAG summarization system against four commercial LLM baselines using both standard NLP and novel educational metrics on a real university lecture corpus.

7.4 Product contribution

The LOA-ESS delivers a production-quality, multi-output educational content generation system integrated into the AuraLearn student dashboard. Students can upload any lecture materials, define their module's learning outcomes once, and receive on-demand: structured summaries with source citations, Bloom's-aligned flashcard decks, LO-mapped quiz sets, and hierarchical mind maps — all from a single retrieval pass over their curriculum content.

7.5 Limitations and threats to validity

Small evaluation corpus: 12 module-week instances across 5 modules is insufficient for statistical conclusions that generalize beyond SLIIT SE modules. Results may not transfer to other disciplines, institutions, or languages.

Researcher-written reference summaries: The reference summaries used for ROUGE/BERTScore computation were written by the researcher, introducing potential bias toward the LOA-ESS output style.

LOCS and BAS keyword sensitivity: The keyword-based and indicator-phrase-based components of LOCS and BAS may reward summaries that use the right words without genuine conceptual coverage.

LLM variability: All LLM-based evaluations (including LLM-as-judge) are subject to generation variability. Results are reported as means across multiple runs, but confidence intervals were not formally computed.

Baseline prompt optimization: The LLM-only baselines were prompted with LOs using a standard template. Expert prompt engineers might achieve higher baseline performance with optimized prompts, narrowing the ERS gap.

7.6 Implications

LO-anchored retrieval is a principled design pattern for any educational RAG application where multiple learning outcomes with different priorities and Bloom's levels must be addressed in a single generation pass.

Bloom's taxonomy provides a practical vocabulary for prompt engineering in educational contexts, enabling fine-grained cognitive depth control without complex fine-tuning.

Educational domain-specific metrics are necessary for evaluating AI-generated educational content; standard NLP metrics systematically underestimate the quality gap between aligned and non-aligned systems.

RAG is essential for large lecture corpora: For modules with extensive lecture materials exceeding LLM context windows, RAG is not merely a quality improvement but a functional necessity.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The LOA-ESS addresses a genuine unmet need in the EdTech market:

Primary target users:

- University students preparing for examinations and needing structured, curriculum-aligned study resources
- Academic institutions seeking to supplement lecture delivery with AI-generated, faculty-approved study materials

Key value proposition:

- For students: Receive exam-focused summaries, flashcards, and quizzes that directly address what the module requires them to know and be able to do — not a generic summary of the lecture text.
- For institutions: Automate the generation of study guides and practice materials that are grounded in the module's formally specified learning outcomes, reducing lecturer workload while ensuring curriculum alignment.

8.2 Market and competitive position

The global EdTech market was valued at USD 254 billion in 2024 (Grand View Research) with a 13.4% CAGR through 2030. The AI-powered learning content generation sub-segment is the fastest-growing category within EdTech, driven by the proliferation of LLMs.

Competitive landscape:

- NotebookLM: Strong document grounding but no LO alignment; free consumer product not designed for institutional deployment.
- Khanmigo (Khan Academy): Interactive AI tutoring with learning alignment for K-12; not designed for university lecture-to-summary workflows.
- Quizlet AI / Anki AI: Flashcard generation from text; no LO alignment or Bloom's differentiation.
- Generic LLMs (ChatGPT, Gemini): Capable but require expert prompting for LO alignment; no persistent retrieval over lecture corpora.

The LOA-ESS is the only system offering: (a) native learning outcome specification; (b) Bloom's-level differentiation; (c) source-cited output; and (d) multi-output generation (summary + flashcards + quiz + mind map) from a single retrieval pass.

8.3 Business or sustainability model

Revenue model options:

- SaaS subscription (institutional): Universities pay per-seat per semester. Includes module setup, LO ingestion, and unlimited generation within usage quotas.
- LMS plugin licensing: White-label LOA-ESS API integrated into Moodle, Canvas, or Blackboard as a native "Generate Study Resources" button, licensed per institution.
- API usage-based pricing: EdTech platforms pay per 1,000 API generation calls.
- Freemium (student-direct): Free tier (5 generations/month) with premium subscription for unlimited use.

Sustainability: The use of open-source infrastructure (Qdrant, PostgreSQL, Redis, FastAPI) eliminates ongoing software licensing costs. The primary running cost is LLM inference — at USD 0.0012 per summary generation (Gemini 2.0 Flash pricing), serving 10,000 students generating 5 summaries per semester costs approximately USD 60/semester. This is commercially viable at institutional scale.

8.4 Scalability, intellectual property and compliance

Scalability**:** The stateless FastAPI microservice is horizontally scalable. Qdrant supports distributed clustering for multi-node scaling. Redis-backed Celery workers can be scaled horizontally for document ingestion. The PostgreSQL database can be moved to a managed cloud service (AWS RDS, Google Cloud SQL) for production scale.

Intellectual property: The LO-anchored hybrid retrieval algorithm, the Bloom's-aware prompt construction framework, the ERS metric suite, and the LOCS/BAS/SGR metric definitions represent novel technical contributions that may be protectable as patents or trade secrets. The specific combination of per-LO retrieval with Bloom's-level prompt differentiation is not found in any published prior work.

Compliance: The system uses only submitted lecture documents (no scraping or external content ingestion). GDPR compliance is maintained by avoiding personal student data storage; only anonymised session tokens are used. Content copyright is the responsibility of the uploading institution.

8.5 Contribution to the Sustainable Development Goals

The LOA-ESS aligns primarily with UN SDG 4 (Quality Education), specifically:

- Target 4.1: Ensure all learners acquire quality education. LOA-ESS provides curriculum-aligned, source-verified study resources that improve the quality of self-directed learning.
- Target 4.4: Substantially increase youth with relevant skills. By providing AI-generated practice quizzes and flashcards, the system supports active skill development aligned with module learning outcomes.

Secondary alignment with SDG 9 (Industry, Innovation and Infrastructure) is present through the novel technical contributions (ERS metric suite, LO-anchored RAG) that advance the state of the art in educational AI infrastructure.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the Learning Outcome-Aware Educational Summarization System (LOA-ESS) — the individual component of the AuraLearn AI-enabled intelligent study assistant (R26-SE-022) — which addresses the gap between generic AI summarization tools and the curriculum-specific needs of university students preparing for examinations.

The LOA-ESS pipeline combines five technical innovations: LO-Anchored Hybrid Retrieval (per-LO dense+sparse RAG with RRF fusion), Cross-Encoder Re-ranking with LO-boost, Bloom's-Aware Prompt Construction (level-specific cognitive instructions per learning outcome), multi-output generation (summary, flashcards, quiz, mind map), and four novel educational evaluation metrics (LOCS, BAS, SGR, ERS).

Evaluation against four commercial baselines on a real university lecture corpus demonstrates that LOA-ESS achieves an ERS of 0.81 — a +0.21 improvement over the best LLM-only baseline — with LOCS of 0.86, BAS of 0.79, and SGR of 0.68. Ablation confirms that both LO-anchored retrieval and Bloom's prompt construction independently and additively contribute to the ERS advantage. User study results confirm that students strongly prefer LOA-ESS outputs on all five evaluated dimensions.

9.2 Achievement of objectives

All stated objectives have been achieved:

Literature reviewed — 18 relevant works synthesized into the gap analysis and conceptual framework

LO-Anchored Hybrid Retrieval designed and implemented

Cross-Encoder Re-ranking with LO boost implemented

Bloom's-Aware Prompt Builder designed and implemented — all six Bloom's level instruction sets

Four novel metrics designed and implemented

Multi-output generation implemented — summary, flashcards, quiz, mind map

Evaluation completed — LOA-ESS vs 4 baselines, 12 evaluation instances, $p < 0.01$ significance

FastAPI microservice deployed — port 8000, Oracle Cloud, 99.6% availability

Frontend integration completed — summary, flashcard, quiz, mind map, and compare pages

Functional tests — all 10 test cases pass

9.3 Individual research and product contributions

Research contributions:

- LO-anchored hybrid retrieval as a novel RAG design pattern for multi-LO educational content generation
- Bloom's-aware prompt engineering framework for cognitive-depth-differentiated LLM generation
- ERS, LOCS, BAS, and SGR — four novel, automated educational evaluation metrics
- First controlled comparison of LO-anchored RAG vs commercial LLM baselines using educational metrics

Product contributions:

- Production-quality LOA-ESS FastAPI microservice (port 8000) with PostgreSQL + Qdrant + Redis + MinIO backend
- Multi-output generation system (summary + flashcards + quiz + mind map) from a single retrieval pass
- Next.js student dashboard pages for all output types with real-time generation and comparison view
- Docker deployment configuration and Oracle Cloud production deployment

9.4 Future directions

Adaptive LO weighting: Weight per-LO retrieval effort by the LO's assessment weight, allocating more retrieval budget to high-stakes learning outcomes.

Streaming generation: Stream the LLM output token-by-token to the frontend to reduce perceived latency for summary generation.

Personalized summarization: Track which LOs each student consistently scores poorly on in quizzes and bias future retrieval toward those LOs.

Multi-language support: Extend the ingestion pipeline to support Sinhala and Tamil language lecture materials for Sri Lankan university deployments.

Larger-scale evaluation: Conduct evaluation on a corpus of 50+ modules across multiple disciplines with independently produced reference summaries and a formal inter-annotator reliability study.

Fine-tuned embedding model: Fine-tune a domain-specific embedding model on SLIIT lecture material pairs to improve retrieval precision over the general-purpose baseline.

Integration with student performance data: Evaluate whether regular use of LOA-ESS-generated study resources produces measurable improvements in module assessment scores in a controlled longitudinal study.

REFERENCES

[1] L. A. Anderson and D. R. Krathwohl (Eds.), *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York: Longman, 2001.

[2] B. S. Bloom (Ed.), *Taxonomy of Educational Objectives: The Classification of Educational Goals, Handbook I: Cognitive Domain*. New York: Longman, 1956.

[3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[4] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in *Proc. 32nd International ACM SIGIR Conference*, 2009, pp. 758–759.

[5] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.

[6] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Proc. ACL Workshop on Text Summarization Branches Out*, 2004.

[7] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in *Proc. ICLR 2020*, 2020.

[8] L. Piro, T. Bianchi, L. Alessandrelli, A. Chizzola, and D. Casiraghi, "MyLearningTalk: An LLM-based intelligent tutoring system," *Lecture Notes in Computer Science*, 2024. DOI: 10.1007/978-3-031-62362-2_39.

[9] Y. Obata and M. Hamada, "Adaptive Retrieval-Augmented Generative Intelligent Tutoring System for Automata Theory," in *Proc. 2025 IEEE MCSoC*, Singapore, 2025, pp. 477–482.

[10] A. T. Neumann, Y. Yin, S. Sowe, S. Decker, and M. Jarke, "An LLM-Driven Chatbot in Higher Education for Databases and Information Systems," *IEEE Trans. Educ.*, vol. 68, no. 1, pp. 103–116, 2025.

[11] A. Sabar, S. Gamage, S. Abishek, C. Lakmal, K. A. D. T. Kulawansa, and P. Perera, "AI-Powered Student Learning System," in *Proc. 8th SLAAI-ICAI*, 2024.

[12] M. Kaouni, F. Lakrami, and O. Labouidya, "Integrating Artificial Intelligence and NLP in E-Learning Platforms," in *Proc. 7th IEEE CiSt*, 2023, pp. 56–63.

[13] M. Sindhu, Sarita, Y. Y. Raghav, and K. Kundu, "Generative AI in Education and its Future Possibilities," in *Proc. 2025 CISES*, 2025, pp. 68–74.

[14] V.-P. Sorescu and D. Andone, "Developing Intelligent Tutors with AI for Digital Education," in *Proc. 2025 IEEE DEMOcon*, 2025.

[15] V. K, M. T. N, S. Nithyashri, A. S. Gopal, and S. R. P, "Dynamic Content Generation for LMS Using Generative AI," in *Proc. 2025 COMP-SIF*, 2025.

[16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP 2019*, 2019.

[17] A. Aher, R. Sawant, R. Patil, S. Sabat, and T. Panchal, "Mindscape: An AI-Powered Study Companion Integrating NLP Algorithms," in *Proc. 3rd ICCSAI*, 2025.

[18] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

GLOSSARY

BAS (Bloom's Alignment Score). A novel metric measuring whether a generated summary addresses each learning outcome at the cognitive depth prescribed by Bloom's Taxonomy, computed by checking for Bloom's-level-specific indicator phrases.

ERS (Educational Relevance Score). The primary composite evaluation metric for LOA-ESS: ERS = 0.40 × LOCS + 0.30 × BAS + 0.20 × SGR + 0.10 × Coherence.

Hybrid Retrieval. A retrieval approach that combines dense vector search (semantic similarity via embeddings) and sparse keyword search (BM25/TF-IDF), fusing results using Reciprocal Rank Fusion.

Learning Outcome (LO). A formally stated specification of what a student should know, understand, or be able to do upon completing a module or unit, expressed in Bloom's Taxonomy terms.

LO-Anchored Retrieval. The novel retrieval strategy in LOA-ESS that executes a separate hybrid retrieval query per learning outcome, ensuring each LO's content is represented in the retrieved context.

LOCS (Learning Outcome Coverage Score). A novel metric measuring the proportion of a module's learning outcomes that are meaningfully addressed in a generated summary, computed as a weighted combination of keyword presence (30%), semantic similarity (40%), and Bloom's depth matching (30%).

LOA-ESS. Learning Outcome-Aware Educational Summarization System — the name of this individual component.

RAG (Retrieval-Augmented Generation). An architecture that augments LLM generation by first retrieving relevant passages from a document corpus and conditioning generation on those passages, reducing hallucination and enabling domain-specific grounding.

SGR (Source Grounding Rate). A novel metric measuring the proportion of substantive claims in a generated summary that include explicit source citations.

APPENDICES

Appendix A — Research instruments or extended results

Evaluation corpus: The complete evaluation corpus (module LO specifications, reference summaries, and raw metric results) is available in evaluation/data/evaluation_corpus/ in the project repository. The corpus covers 12 module-week instances across five SLIIT Software Engineering modules.

Full per-instance results: Per-instance ROUGE, BERTScore, LOCS, BAS, SGR, and ERS scores for all 12 evaluation instances and all 5 systems (LOA-ESS + 4 baselines) are stored in evaluation/results/full_results.csv. The mean values reported in Table 6.1 are reproducible by running python evaluation/scripts/aggregate_results.py.

Ablation raw data: Raw ERS scores for each of the 4 ablation configurations across all 12 evaluation instances are stored in evaluation/results/ablation_results.csv.

Appendix B — Academic integrity and AI use disclosure

Generative AI tools were used for the following limited academic-support activities during the preparation of this thesis: language refinement and clarity improvement of draft paragraphs; formatting assistance in aligning content with the SLIIT individual thesis template; and code documentation string generation.

All research design decisions, algorithm specifications, metric definitions, experiment designs, evaluation results, system architecture decisions, interpretations, and conclusions presented in this thesis are the original work of the author (Vijithapala T.G.K.G, IT22187178). All cited experimental results are computed from the actual implemented system on real evaluation data. The author takes full responsibility for the accuracy and integrity of all content in this thesis.

End of Individual Thesis

Vijithapala T.G.K.G — IT22187178

Sri Lanka Institute of Information Technology — October 2026